

Operativni sistemi

Deljena memorija (Laboratorijska vežba III-2)

Zadatak 1

Korišćenjem programskog jezika C napisati UNIX/Linux program koji se deli u dva procesa (proizvođač - potrošač) koji komuniciraju koristeći ograničeni kružni bafer u deljivoj memoriji. Kružni bafer je veličine 10 celih brojeva. Ovi procesi sinhronizuju svoje aktivnosti (upis u bafer i čitanje iz bafera) koristeći semafore.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>

#define FULL_KEY 100001
#define EMPTY_KEY 100002
#define MUTEX_KEY 100003
#define MEM_KEY 100004
#define N 10

union semun
{
    int val;
    struct semid_ds *buf;
    ushort *array;
    struct seminfo * __buf;
    void * __pad;
};

int main()
{
    //celobrojne reference na semafore i deljenu memoriju
    int fullid, emptyid, mutexid;
    int memid;
    int * shmem;//pokazivac za mapiranje deljene memorije u adresni procesa
    int i;
    struct sembuf lock = {0, -1, 0};
    struct sembuf unlock = {0, 1, 0};
    union semun opts;

    //kreiranje i inicijalizacija semafora FULL
    fullid = semget(FULL_KEY, 1, IPC_CREAT | 0666);
    if (fullid < 0)
    {
        printf("Greska prilikom kreiranja semafora FULL\n");
        exit(1);
    }
    opts.val = 0;
    if (semctl(fullid, 0, SETVAL, opts) < 0)
    {
        printf("Greska prilikom inicijalizacije semafora FULL\n");
        exit(1);
    }
}
```

```

}
//kreiranje i inicijalizacija semafora empty
emptyid = semget(EMPTY_KEY, 1, IPC_CREAT | 0666);
if (emptyid < 0)
{
    printf("Greska prilikom kreiranja semafora EMPTY\n");
    exit(1);
}
opts.val = N;
if (semctl(emptyid, 0, SETVAL, opts) < 0)
{
    printf("Greska prilikom inicijalizacije semafora EMPTY\n");
    exit(1);
}
//kreiranje i inicijalizacija semafora MUTEX
mutexid = semget(MUTEX_KEY, 1, IPC_CREAT | 0666);
if (mutexid < 0)
{
    printf("Greska prilikom kreiranja semafora MUTEX\n");
    exit(1);
}
opts.val = 1;
if (semctl(mutexid, 0, SETVAL, opts) < 0)
{
    printf("Greska prilikom inicijalizacije semafora MUTEX\n");
    exit(1);
}
//kreira se deljiva memorija velicine N celih brojeva
memid = shmget(MEM_KEY, N * sizeof(int), IPC_CREAT | 0666);
if (memid < 0)
{
    printf("Greska prilikom kreiranja deljive memorije\n");
    exit(1);
}
if (fork() == 0)
{
    //izvršava proces dete koje funkcioniše kao proizvođač
    //pozicija u baferu u kojoj će se obaviti sledeći upis
    int next_write = 0;
    //mapira se deljiva memorija u adresni prostor procesa
    shmem = shmat(memid, NULL, 0);
    if (shmem == NULL)
    {
        printf("Greska prilikom mapiranja deljive memorije\n");
        exit(1);
    }
    for (i = 0; i < 20; i++)
    {
        //standardna sekvenca operacija za proces proizvođač
        semop(emptyid, &lock, 1);
        semop(mutexid, &lock, 1);
        //upis podataka u deljivu memoriju
        int p = rand() % 100;
        shmem[next_write] = p;
        next_write = (next_write + 1) % N;
        printf("Generisana vrednost je %d\n", p);
        semop(mutexid, &unlock, 1);
        semop(fullid, &unlock, 1);
        sleep(2);
    }
}

```

```

    }
    //izbacujemo deljenu memoriju iz adresnog prostora
    shmdt(shmem);
}
else
{
    //izvrsava proces roditelj koji funkcioniše kao potrosac
    //pozicija u baferu sa koje ce se obaviti sledece citanje
    int next_read = 0;
    //mapiraa se deljiva memorija u adresni prostor procesa
    shmem = shmat(memid, NULL, 0);
    if (shmem == NULL)
    {
        printf("Greska prilikom mapiranja deljive memorije\n");
        exit(1);
    }
    for (i = 0; i < 20; i++)
    {
        //standardna sekvenca operacija za proces potrosac
        semop(fullid, &lock, 1);
        semop(mutexid, &lock, 1);
        //upis podataka u deljivu memoriju
        printf("Ocitana vrednost je %d\n", shmem[next_read]);
        next_read = (next_read + 1) % N;
        semop(mutexid, &unlock, 1);
        semop(emptyid, &unlock, 1);
        sleep(5);
    }
    //izbacujemo deljenu memoriju iz adresnog prostora
    shmdt(shmem);
    //uklanjamo sve semafore i deljenu memoriju
    semctl(fullid, 0, IPC_RMID, 0);
    semctl(emptyid, 0, IPC_RMID, 0);
    semctl(mutexid, 0, IPC_RMID, 0);
    shmctl(memid, IPC_RMID, 0);
}

return 0;
}

```

Zadatak 2

Korišćenjem programskog jezika C kreirati tri Linux procesa koji za međusobnu komunikaciju koriste deljenu memoriju veličine 1024 bajtova. Prvi proces popunjava prvih 512 bajtova deljive memorije slučajno izabranim slovima (u opsegu a-z). Nakon toga, drugi proces popunjava poslednjih 512 bajtova deljene memorije, proizvoljno izabranim ciframa. Pošto i drugi proces završi generisanje podataka, treći proces kompletan sadržaj deljene memorije upisuje u datoteku. Ova sekvenca akcija se periodično ponavlja svakih 15 s.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

```

```

#include <sys/shm.h>

#define PRVI_KEY 10001
#define DRUGI_KEY 10002
#define TRECI_KEY 10003
#define MEM_KEY 10004
#define N 1024

union semun
{
    int val;
    struct semid_ds *buf;
    ushort *array;
    struct seminfo * __buf;
    void * __pad;
};

int main()
{
    int sem1, sem2, sem3, memid; //reference semafora i deljive memorije
    int i, j;
    FILE * f;
    //pokazivac na niz bajtova koji se koristi za mapiranje deljive memorije
    char * shmem;
    //pomocne strukture za inicijalizaciju i operacije nad semaforom
    struct sembuf lock = {0, -1, 0};
    struct sembuf unlock = {0, 1, 0};
    union semun opts;
    //kreiramo i inicijalizujemo semafore
    //inicijalno se postavljaju na vrednosti 1-0-0 jer zapocinje da se izvršava
    //proces P1 pa nakon njega proces P1 pa tek onda P3
    sem1 = semget(PRVI_KEY, 1, IPC_CREAT | 0666);
    sem2 = semget(DRUGI_KEY, 1, IPC_CREAT | 0666);
    sem3 = semget(TRECI_KEY, 1, IPC_CREAT | 0666);
    opts.val = 1;
    semctl(sem1, 0, SETVAL, opts);
    opts.val = 0;
    semctl(sem2, 0, SETVAL, opts);
    opts.val = 0;
    semctl(sem3, 0, SETVAL, opts);
    //kreiramo deljenu memoriju
    memid = shmget(MEM_KEY, N * sizeof(char), IPC_CREAT | 0666);
    if (fork() == 0)
    {
        //proces P1 koji generise slova a-z i semsta ih u prvih 512 bajtova
        //mapiramo deljenu memoriju
        shmem = (char *)shmat(memid, NULL, 0);
        for (i = 0; i < 10; i++)
        {
            semop(sem1, &lock, 1); //P(S1)
            //upisujemo slova u prvih 512 bajtova
            for (j = 0; j < N/2; j++)
                shmem[j] = 'a' + rand() % ('z'-'a' + 1);
            semop(sem2, &unlock, 1); //V(S2)
        }
        //izbacujemo delejnu memoriju iz adresnog prostora
        shmdt(shmem);
        return 0;
    }
}

```

```

if (fork() == 0)
{
    //proces P2 koji generise cifre i smesta ih u poslednjih 512 bajtova
    //mapiramo deljenu memoriju
    shmem = (char *)shmat(memid, NULL, 0);
    for (i = 0; i < 10; i++)
    {
        semop(sem2, &lock, 1); //P(S2)
        //upisujemo slova u prvih 512 bajtova
        for (j = N/2; j < N; j++)
        shmem[j] = '0' + rand() % ('9' - '0' + 1);
        semop(sem3, &unlock, 1); //V(S3)
    }
    //izbacujemo delejnu memoriju iz adresnog prostora
    shmdt(shmem);
    return 0;
}
//roditeljski proces P3 koji stampa sadrzaj deljene memorije u datoteku
//mapiramo deljenu memoriju
shmem = (char *)shmat(memid, NULL, SHM_RDONLY);
for (i = 0; i < 10; i++)
{
    semop(sem3, &lock, 1); //P(S3)
    //sadrzaj deljene memorije upisujemo u datoteku
    char buff[N + 1];
    //koristimo funkciju strncpy jer u deljenoj memoriji na kraju nemamo /0
    //kao kraj stringa
    strncpy(buff, shmem, N);
    buff[N] = '\0';
    //sadrzaj bafera upisujemo u datoteku
    f = fopen("sadrzaj", "a");
    fprintf(f, "%s\n", buff);
    fclose(f);
    semop(sem1, &unlock, 1); //V(S1)
}
//izbacujemo delejnu memoriju iz adresnog prostora
shmdt(shmem);
semctl(sem1, 0, IPC_RMID, opts);
semctl(sem2, 0, IPC_RMID, opts);
semctl(sem3, 0, IPC_RMID, opts);
shmctl(memid, IPC_RMID, 0);
return 0;
}

```

Zadatak 3

Korišćenjem programskog jezika C napisati Linux koji simulira sinhronizacioni problem poznat pod nazivom Večera filozofa.

```

//datoteka III-1-3.h

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>

```

```

#define SEMAFORI_KEY 10001
#define MUTEX_KEY 10002
#define STATUS_KEY 10004

//broj filozofa
#define N 5

//enum koji definise moguće vrednosti za status u kome se filozof nalazi
enum status {THINKING, HUNGRY, EATING};

union semun
{
    int val;
    struct semid_ds *buf;
    ushort *array;
    struct seminfo * __buf;
    void * __pad;
};

```

```

//datoteka III-1-3.c

#include "III-1-3.h"

int main()
{
    int s, mutex, statusi;
    int i;
    enum status * shmem;

    union semun opts;

    //kreiramo i inicijalizujemo niz od N semafora. po jedan za svakog filozofa.
    //semafori se koriste za blokiranje filozofa u situaciji kada ne može da
    pribavi dve viljuske
    //koristimo mogućnost da System V semafor možemo da posmatramo kao niz
    semafora
    s = semget(SEMAFORI_KEY, N, IPC_CREAT | 0666);

    //sve semafore koji su vezani za filozofe inicijalizujemo na 1
    opts.val = 1;
    for (i = 0; i < N; i++)
        semctl(s, i, SETVAL, opts);

    //kreiramo i inicijalizujemo MUTEX semafor
    //obezbedjuje medjusobno isključenje filozofa kada proveravaju dostupnost
    resursa odnosno viljusaka
    mutex = semget(MUTEX_KEY, N, IPC_CREAT | 0666);

    opts.val = 1;
    semctl(mutex, 0, SETVAL, opts);

    //kreiramo i inicijalizujemo deljivu memoriju u kojoj će se cuvati
    informacije o statusu svakog filozofa
    statusi = shmget(STATUS_KEY, N * sizeof(enum status), IPC_CREAT | 0666);

    //mapiramo deljenu memoriju u adresni prostor procesa
    shmem = (enum status *)shmat(statusi, NULL, 0);
}

```

```

//inicijalizujemo status svih filozofa na THINKING
for (i = 0; i < N; i++)
    shmem[i] = THINKING;

//izbacujemo deljivu memoriju iz adresnog prostora jer nam vise nije potrebna
shmdt(shmem);

//pokrecemo N filozofa
for (i = 0; i < N; i++)
{
    if (fork() == 0)
    {
        char buff[2];
        sprintf(buff, "%d", i);

        execl("III-1-3a", "III-1-3a", buff, NULL);
    }
}

//cekamo da se svi filozofi zavrse
for (i = 0; i < N; i++)
    wait(NULL);

//brisemo sve kreirane objekte
semctl(s, 0, IPC_RMID, opts);
semctl(mutex, 0, IPC_RMID, opts);

shmctl(statusi, IPC_RMID, 0);

return 0;
}

```

```

//datoteka III-1-3a.c

#include "III-1-3.h"

//globalne promenljive koje treba da budu vidljive u svim funkcijama
int s, mutex, statusi;
enum status * shmem;
struct sembuf lock = {0, -1, 0};
struct sembuf unlock = {0, 1, 0};

void think(int no);
void take_forks(int no);
void put_forks(int no);
void eat(int no);
void test(int no);

int main(int argc, char * argv[])
{
    int i;
    int no = atoi(argv[1]);

```

```

//pribavljamo referencu na semafore S koji predstavljaju filozofe
s = semget(SEMAFORI_KEY, N, 0666);

//pribavljamo referencu na MUTEX semafor
mutex = semget(MUTEX_KEY, N, 0666);

//pribavljamo referencu na delejnu memoriju u kojoj se cuvaju statusi
filozofa
statusi = shmget(STATUS_KEY, N * sizeof(enum status), 0666);

//mapiramo deljenu memoriju u adresni prostor procesa
shmem = (enum status *)shmat(statusi, NULL, 0);

for (i = 0; i < 10000; i++)
{
    think(no);
    take_forks(no);
    eat(no);
    put_forks(no);
}

//izbacujemo deljenu memoriju iz adresnog prostora
shmdt(shmat);

return 0;
}

void think(int no)
{
    //razmislja najvise 10s
    printf("Filozof %d razmislja\n", no);
    sleep(rand() % 11);
}

void eat(int no)
{
    //jede najvise 5s
    printf("Filozof %d vecera\n", no);
    sleep(rand() % 6);
}

void take_forks(int no)
{
    //zakljucavamo MUTEX
    lock.sem_num = 0;
    semop(mutex, &lock, 1);

    //od mnogo razmisljanja se ogladni
    shmem[no] = HUNGRY;

    //filozof pokusava da pribavi viljuske
    test(no);

    //otkljucavamo MUTEX
    unlock.sem_num = 0;
    semop(mutex, &unlock, 1);
}

```

```

    //P operacija nad odgovarajucim semaforom S
    //ukoliko viljuske nisu pribavljene filozof se blokira
    lock.sem_num = no;
    semop(s, &lock, 1);
}

void test(int no)
{
    if (shmem[no] == HUNGRY
        && shmem[(no + N -1) % N] != EATING //levi filozof ne sme da jede u
        datom trenutku
        && shmem[(no + 1) % N] != EATING //desni filozof ne sme da jede u datom
        trenutku
    )
    {
        //ako je sve OK odnosno ima na raspolaganju dve viljuske pocinje da
        jede
        shmem[no] = EATING;

        //signalizira se odgovarjauci semafor kako se funkcija take_forks ne bi
        blokirala
        unlock.sem_num = no;
        semop(s, &unlock, 1);
    }
}

void put_forks(int no)
{
    //zakljucavamo MUTEX
    lock.sem_num = 0;
    semop(mutex, &lock, 1);

    //posle jela filozof ponovo moze da misli
    shmem[no] = THINKING;

    //signalizira svojim susedima da je oslobodio viljusku
    test((no + N -1) % N);
    test((no + 1) % N);

    //otkljucavamo MUTEX
    unlock.sem_num = 0;
    semop(mutex, &unlock, 1);
}

```

Zadatak 4

Korišćenjem programskog jezika C kreirati dva Linux procesa. Prvi proces kreira u deljenom memorijskom segmentu matricu dimenzija MxN. Proces 2 u svakoj koloni matrice pronalazi maksimalni element i štampa ga na ekranu.

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>

#define N 4
#define M 5
#define MEM_KEY 1122
#define SEM_KEY 3344

union semun{
    int val;
    struct semid_ds *buf;
    ushort *array;
};

int main()
{
    int i, j;
    int semid;
    union semun semopts;
    struct sembuf sem_lock = {0, -1, 0};
    struct sembuf sem_unlock = {0, 1, 0};
    semid = semget((key_t)SEM_KEY, 1, 0666 | IPC_CREAT);
    semopts.val = 0;
    semctl(semid, 0, SETVAL, semopts);
    if (fork() != 0)
    {
        int shm_id = shmget(MEM_KEY, M*N*sizeof(int), IPC_CREAT | 0666);
        int* shm_ptr = shmat(shm_id, NULL, 0);
        for(i=0 ; i<N ; i++)
            for(j=0 ; j<M ; j++)
            {
                shm_ptr[i*M+j] = rand() % 100;
                printf("Upisujem na poziciju %d,%d broj %d\n",
                    i, j, shm_ptr[i*M+j]);
            }
        semop(semid, &sem_unlock, 1);
        shmdt(shm_ptr);
        int status;
        wait(&status);
        shmctl(shm_id, IPC_RMID, NULL);
        semctl(semid, 0, IPC_RMID, NULL);
    }
    else
    {
        int shm_id = shmget(MEM_KEY, M*N*sizeof(int), IPC_CREAT | 0666);
        int* shm_ptr = shmat(shm_id, NULL, 0);
        semop(semid, &sem_lock, 1);
        for(i=0 ; i<N ; i++)
        {
            int maxel = 0;
            for(j=0 ; j<M ; j++)
            {
                if (shm_ptr[i*M+j] > maxel)
                    maxel = shm_ptr[i*M+j];
            }
            printf("Maksimalni element kolone %d je: %d\n", i, maxel);
        }
    }
}

```

```

    }
    shmdt(shm_ptr);
}
}

```

Zadatak 5

Korišćenjem programskog jezika C kreirati program koji se deli u dva procesa. Proces roditelj čita sa tastature stringove i koristeći deljenu memoriju prosleđuje ih procesu detetu. Proces dete primljene stringove upisuje u datoteku stringovi.txt. Kada korisnik unese string “KRAJ” komunikacija se prekida i procesi se završavaju.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/sem.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

#define MEM_KEY 1122
#define PROC_A_KEY 10101
#define PROC_B_KEY 10102

union semun{
    int val;
    struct semid_ds *buf;
    ushort *array;
};

int main()
{
    int i, procaid, procbid;
    union semun semopts;
    struct sembuf sem_lock = {0, -1, 0};
    struct sembuf sem_unlock = {0, 1, 0};
    procaid = semget((key_t)PROC_A_KEY, 1, 0666 | IPC_CREAT);
    procbid = semget((key_t)PROC_B_KEY, 1, 0666 | IPC_CREAT);
    semopts.val = 1;
    semctl(procaid, 0, SETVAL, semopts);
    semopts.val = 0;
    semctl(procbid, 0, SETVAL, semopts);
    if (fork() != 0)
    {
        char line[80];
        int shm_id = shmget(MEM_KEY, 80*sizeof(char), IPC_CREAT | 0666);
        char* shm_ptr = shmat(shm_id, NULL, 0);
        while(strcmp(line, "KRAJ") != 0)
        {
            semop(procaid, &sem_lock, 1);
            printf("Unesite string: ");
            gets(line);
            memcpy(shm_ptr, line, 80);
            semop(procbid, &sem_unlock, 1);
        }
        shmdt(shm_ptr);
    }
}

```

```

    int status;
    wait(&status);
    shmctl(shm_id, IPC_RMID, NULL);
    semctl(procaid, 0, IPC_RMID, NULL);
    semctl(procbid, 0, IPC_RMID, NULL);
}
else
{
    char line[80];
    int shm_id = shmget(MEM_KEY, 80*sizeof(char), IPC_CREAT | 0666);
    char* shm_ptr = shmat(shm_id, NULL, 0);
    while(strncmp(line, "KRAJ") != 0)
    {
        semop(procbid, &sem_lock, 1);
        memcpy(line, shm_ptr, 80);
        printf("Iz deljene memorije procitano: %s\n", line);
        fflush(stdout);
        semop(procaid, &sem_unlock, 1);
    }
    shmdt(shm_ptr);
}
}

```

Zadatak 6

Korišćenjem programskog jezika C kreirati program koji kopira zadatu datoteku. Prvi argument komandne linije je naziv originalne datoteke, a drugi argument je naziv nove datoteke, koja će biti kopija prve. Kopiranje se obavlja tako što se program deli u dva procesa, prvi proces čita određeni broj bajtova iz datoteke i šalje ih drugom procesu. Drugi proces bajtove iz deljene memorije upisuje u novu datoteku.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/sem.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <fcntl.h>

#define MEM_KEY 1122
#define PROC_A_KEY 10101
#define PROC_B_KEY 10102
#define BUF_SIZE 80

union semun{
    int val;
    struct semid_ds *buf;
    ushort *array;
};

struct rdstruct{
    char nch;
    char buf[BUF_SIZE];
};

```

```

int main(int argc, char* argv[])
{
    int i, procaid, procbid;
    union semun semopts;
    struct sembuf sem_lock = {0, -1, 0};
    struct sembuf sem_unlock = {0, 1, 0};

    procaid = semget((key_t)PROC_A_KEY, 1, 0666 | IPC_CREAT);
    procbid = semget((key_t)PROC_B_KEY, 1, 0666 | IPC_CREAT);
    semopts.val = 1;
    semctl(procaid, 0, SETVAL, semopts);
    semopts.val = 0;
    semctl(procbid, 0, SETVAL, semopts);
    printf("argv[1]: %s, argv[2]: %s\n", argv[1], argv[2]);

    if (fork() != 0)
    {
        int shm_id = shmget(MEM_KEY, sizeof(struct rdstruct), IPC_CREAT | 0666);
        struct rdstruct *shm_ptr = shmat(shm_id, NULL, 0);

        int fd = open(argv[1], O_RDONLY);
        shm_ptr->nch = BUF_SIZE;

        while(shm_ptr->nch == BUF_SIZE)
        {
            semop(procaid, &sem_lock, 1);
            shm_ptr->nch = read(fd, shm_ptr->buf, BUF_SIZE);
            printf("Upisao u shm %d bajtova: %s\n", shm_ptr->nch, shm_ptr->buf);
            fflush(stdout);
            semop(procbid, &sem_unlock, 1);
        }

        close(fd);
        shmdt(shm_ptr);
        printf("Proces roditelj zavrrio, ceka dete!\n");
        fflush(stdout);

        int status;
        wait(&status);

        shmctl(shm_id, IPC_RMID, 0);
        semctl(procaid, IPC_RMID, 0);
        semctl(procbid, IPC_RMID, 0);
        close(fd);

        printf("Proces roditelj sacekao dete, izlazi!\n");
        fflush(stdout);
    }
    else
    {
        int shm_id = shmget(MEM_KEY, sizeof(struct rdstruct), IPC_CREAT | 0666);
        struct rdstruct *shm_ptr = shmat(shm_id, NULL, 0);

        int fd = open(argv[2], O_CREAT | O_WRONLY | O_TRUNC, 00777);
        int n = BUF_SIZE;

        while(n == BUF_SIZE)
    }
}

```

```
{
    semop(procbid, &sem_lock, 1);
    write(fd, shm_ptr->buf, shm_ptr->nch);
    n = shm_ptr->nch;
    printf("Procitao iz shm %d bajtova: %s\n", shm_ptr->nch, shm_ptr->buf);
    fflush(stdout);
    semop(procaid, &sem_unlock, 1);
}
close(fd);
shmdt(shm_ptr);

printf("Proces dete zavrsio!\n");
fflush(stdout);
}
```